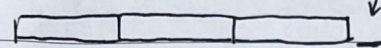


1. 位运算见 chapter 3

2.



见 T3-1

SEEK-END

记得 fread 后光标会向后移动

★看了 fread 写了答案后再更新位置!

3. 文件操作见 chapter 4

r: 读

文件必写

w: 清空再写

a: 只往后写

t: 读写都有

b: 二进制

} 不可 create

空结构体初始化

the function reads beyond the valid memory region of the variable. the behavior is undefined and may result in writing garbage data into the file.

Create storage on the disk in which the file will be stored

make it possible to determine if a record contains data.

4. 见 mock 错题整理

成功读入几个变量 返回几 (eg=0)

5. 文件读取

} 读不匹配 返回已成功读入的个数

文本文件 } fscanf(indata, "%d %*c", &a);

} input 结束 返回 EOF(-1)

fprintf(indata, "%d", a);

读到 'n', 立即 stop 并把 'n' 也一起存进去

fgets(arr, n, indata);

读满 n-1 个字符, 预留 '\0'

失败返回 NULL, 成功返回 arr

遇 EOF

二进制文件

fread(arr, sizeof(...), 1, indata);

fwrite(arr, sizeof(...), 1, outdata);

} 返回成功读取个数

都可用

fseek(indata, (...-1)*sizeof(...), SEEK-SET);

stdin = gets(arr); 遇 'n' 停止, 自动去 'n' 并补 '\0'

一些文件需处理的点:

只是一个普通字符 '所谓' 下一行; 只是不同函数处理它不同

① 文件与 'n' 有关

文件本质: 连续字符流 eg. hello 123 (n) world 456 1n

fgets: 遇 'n' 会停止 (并读进去)

fscanf: 通常会跳过空白 (包括 'n')

② fgets 从当前文件指针开始读, 前面空格 'n' 需处理干净

③ %*c 需用在

(a) 12-A %d %*c %c (fscanf 时)

(c) %s 后有 %c

(b) 123-hello → 先 fscanf 后 fgets

123 1n hello ↗

%*c 的作用是丢弃当前缓冲区中的一个字符, 避免它被后续不会跳过空白的函数 (eg. fgets() 或 %c) 误读

6. `char ** p = calloc(16, sizeof(char*));`

现在的内存结构是: `p` 是一个"指针数组" (16个元素)

每个元素是 `char*`

∴ 可访问 `p[0] ~ p[15]` 而 `p[i]` 还未初始化 ∴ 需 `p[i] = calloc(9, sizeof(char));`

三维数组

`char *** p = calloc(x, sizeof(char**));`

`for(int i=0; i<x; i++)`

`p[i] = calloc(y, sizeof(char*));`

`for(int j=0; j<y; j++)`

`p[i][j] = calloc(z, sizeof(char));`

9

2

是否跳过前导空白

`scanf(" %s", s)`

`gets`

`fgets`

✓

✗

✗

`\n`

✗ 读到就停止, 不读进去

✗ 不读进去

✓ 读进去

`\n` 是否留缓冲区

✓

✗ (丢弃)

✗ (被读走)

是否自动加10

✓

✓

✓

停止条件

第一个空白符

`\n` 或 `EOF`

`\n` 或读满 `n-1` 或 `EOF`

`%ns` 最多读 `n` 个字符 (不包含 `'\0'`)

数组开 `n`

∴ 数组开 `n+1`

8. `%` `%%`

`\`

`\\`

→ 写反

`"`

`\"`

想 output 的字符